

Regex Basics

Pattern	Meaning	Example
^	Start of line	^203\.0\.113\.
\$	End of line	closed\$
.	Any single character	r.ot
\.	A literal dot (escaped)	10\.0\.0\.5
*	0 or more of the previous	ab*c
+	1 or more of the previous (needs -E /\+)	[0-9]+
?	0 or 1 of the previous (needs -E /\?)	colou?r
{n,m}	Between n and m repeats	[0-9]{1,3}
[abc]	Character class: any one of a, b, c	[A-Za-z0-9]
(a b)	Alternation: a OR b (needs -E)	union select
(...)	Capture group, referenced as \1 \2...	(SEQ=[0-9]+)

strings

strings file

Prints every run of 4+ printable characters in a file. First move on anything binary: packet captures, memory dumps, corrupted logs.

```
$ strings frame_capture.bin

-n <len> sets the minimum run length, e.g. strings -n 8 to cut out short noise.
```

grep

grep 'pattern' file

Prints every line matching a pattern.

```
$ grep 'Failed password' auth.log
$ grep -i 'root' auth.log          # ignore case
$ grep -v 'AUTH_OK' auth.log      # invert match
$ grep -c 'AUTH_FAIL' auth.log    # count matches
```

grep -E 'pattern' file

Extended regex: enables | + ? {} () without backslash-escaping them.

```
$ grep -Ei "union.*select|or 1=1" \
    access.log
$ grep -oE "SEQ=[0-9]+" file.log # -o
    prints ONLY the matched text
```

sed

sed 's/find/replace/' file

Stream editor: substitute text. Add a trailing /g to replace every match on the line, not just the first.

```
$ echo 'user=jdoe' | sed 's/user=/USER: /'
USER: jdoe

$ sed -n '/AUTH_FAIL/p' auth.log
# -n + /pat/p = act like grep
```

sed -E 's/(a)(b)/\1 \2/'

Capture groups. -E lets () work without escaping; \1, \2... refer to what each group matched.

```
$ echo 'SEQ=03;PAYLOAD=xyz' | sed -E \
    's/SEQ=([0-9]+);PAYLOAD=(.*)/\1 \2/'
03 xyz
```

awk

awk '{print \$N}' file

Splits each line on whitespace into columns: \$1, \$2, \$3... \$0 is the whole line, \$NF is always the LAST field.

```
$ awk '{print $4}' auth_events.log
203.0.113.44

$ awk '{print $1, $4}' auth_events.log
```

awk 'condition {action}'

Only runs the action on lines where the condition is true - a regex match on the whole line, or an exact column comparison.

```
$ awk '/AUTH_FAIL/{print $4}' auth.log
$ awk ' $3=="AUTH_FAIL"{print $4}' auth.log
$ awk -F',' '{print $2}' file.csv # -F sets
    a custom field delimiter
```

Chaining & Counting

cmd1 | cmd2 | cmd3

Pipe the output of one command into the next. sort + uniq -c + wc -l are the glue that turns matches into answers.

```
$ cut -d' ' -f4 auth_events.log # -f4 = 4th
    field, split on the -d delimiter
$ sort | uniq -c # group + count duplicates
$ sort -rn # rank numbers, highest first
$ wc -l # count lines = count results
```

```
# rank the noisiest IPs
$ awk '{print $4}' auth_events.log \
    | sort | uniq -c | sort -rn | head
46 203.0.113.44

# turn a search into a number
$ grep 'user=jdoe' audit.log \
    | grep 'cmd="ping' | wc -l
3
```

Common Log-Hunting Patterns

What	Pattern (-E)
IPv4 address	[0-9]{1,3}(\.[0-9]{1,3}){3}
Timestamp (ISO-ish)	[0-9]{4}-[0-9]{2}-[0-9]{2}
Email address	[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+
Quoted string	"[^"]*"
Failed auth (SSH)	Failed password AUTH_FAIL
SQL injection tells	union.*select ' or '1'='1 --